

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

COOCH BEHAR GOVERNMENT ENGINEERING COLLEGE

Affiliated to Maulana Abul Kalam Azad University of Technology (MAKAUT), West Bengal

Harinchawra, P.O. Ghughumari, Cooch Behar, West Bengal 736170, India

RF Signal Monitoring System

RF Signal Monitoring, Real-Time Data Acquisition, AI-Based Anomaly Detection
and Geographic Signal Monitoring (2.4 GHz ISM band)

B.Tech Final Year Major Project — Technical Workflow, System Architecture
and Implementation Progress Report (for mentor review)

Project Details

Academic Item	Details
Institution	Cooch Behar Government Engineering College
Department	Electronics and Communication Engineering
Affiliating University	Maulana Abul Kalam Azad University of Technology (MAKAUT), West Bengal
Degree & Track	Bachelor of Technology in ECE — Final Year Major Project
Academic Session	2026–2027 (12-week plan, 1 Oct – 23 Dec 2026)
Report Date	9 October 2026 (Week 2 of 12)
Live Dashboard	rf-signal-monitoring-system.vercel.app
Project Mentor	Prof. Rajib Das, Assistant Professor, Dept. of ECE
Technical Support	Mr. Soumik Sarkar, Lab Technician, Dept. of ECE
Student Project Team	Anupam Dutta (Roll: 34900323003) Aushi Sarkar (Roll: 34900323011) Joydip Das (Roll: 34900323020) Sarupya Guha (Roll: 34900323046) Sohan Ghosh (Roll: 34900323050)

TABLE OF CONTENTS

Executive Summary	3
1 Description	3
1.1 What we are building and why	3
1.2 Core engineering objectives	3
2 Components & Technologies	4
2.1 Hardware bill of materials	4
2.2 Software stack	4
3 Hardware Architecture and Circuit Diagram	5
3.1 Hardware components, purpose and pin connections	6
3.2 Pin interconnection	7
3.3 Subsystem summary: hardware front-end	7
4 Workflow	7
4.1 End-to-end data flow	7
4.2 Signal processing chain	8
5 Architecture Diagrams	8
5.1 Layered system architecture	8
5.2 Frontend and backend architecture with technology stack	8
5.3 Software module interaction	9
6 Embedded Firmware and Device Workflow	9
6.1 Subsystem summary: firmware	10
7 Backend and API Architecture	10
7.1 Pipeline	10
7.2 REST endpoints	10
7.3 Subsystem summary: backend	10
8 Database Architecture	11
9 RF Processing and Machine Learning	11
10 Frontend Dashboard Architecture	12
11 Simulation and Real-Hardware Integration	14
11.1 Simulation scenarios	15
11.2 Hardware bring-up stages	15
12 Deployment Architecture	15
13 Testing and Validation	15
14 Execution Plan	16
14.1 Month 1: Foundation and bench verification (Weeks 1-4, 1-28 Oct)	16
14.2 Month 2: Firmware, cloud and ML (Weeks 5-8, 29 Oct – 25 Nov)	16
14.3 Month 3: Integration, field trials and defence (Weeks 9-12, 26 Nov – 23 Dec)	17
15 Risk Analysis and Mitigation	17
16 Current Progress, Pending Work, and Mentor Review Points	17
16.1 Progress summary as of 9 October 2026	17
16.2 Technical points for the mentor	18
17 Next Steps	18

EXECUTIVE SUMMARY

The **RF Signal Monitoring System** (2.4 GHz ISM band) is a low-cost, portable platform that measures broadband 2.4 GHz signal power, tags each reading with GPS position, analyses it with an unsupervised machine-learning model and shows it on a live web dashboard. This report explains the workflow and architecture and states clearly what is finished and what is planned for each week of the 12-week schedule.

End-to-End Pipeline:

- **RF Front-End:** 50 Ω SMA antenna $\rightarrow$ **SPF5189Z** LNA (+13.7 dB) $\rightarrow$ **AD8318** logarithmic detector (−60 to 0 dBm).
- **Edge Node:** **ESP32-WROOM-32** samples the detector on GPIO34 (12-bit ADC, 64-sample burst averaging), reads a **u-blox NEO-7M** GPS over UART2, drives an **SSD1306** OLED and posts JSON over Wi-Fi/HTTPS.
- **Cloud Backend:** **Python 3.12 / Flask 3.0** REST API (Render, Unicorn) with Bearer-token device authentication and a dual SQL engine (SQLite locally, PostgreSQL in the cloud).
- **Analytics & ML:** NumPy signal processing (20-sample moving average, rate of change, SNR) and a pure-NumPy **Isolation Forest** (100 trees).
- **Dashboard:** **React 19 + Vite 8** single-page app on Vercel with Recharts live charts and a Leaflet/OpenStreetMap coverage map. **Live:** <https://rf-signal-monitoring-system.vercel.app>
- **Simulator:** physics-based testbench (log-distance path loss, wall loss, interference) with 6 scenarios, used before hardware is attached.

Where we are (9 Oct 2026, Week 2): Week 1 (requirements, BOM, datasheets) is **Completed**. Per the plan audit, the simulator (with 30 passing tests), Flask API (16 endpoints), database layer, ML module, signal processing and the React dashboard are already implemented — ahead of their scheduled weeks — and the dashboard is deployed on Vercel at the live link above. The physical circuit assembly is **In progress**; ESP32 firmware, GPS integration, bench calibration, field trials and the final viva are **Planned** for Weeks 3–12.

Items for mentor attention are collected in Section 16.

1. DESCRIPTION

1.1 What we are building and why

The 2.4 GHz ISM band is shared by Wi-Fi, Bluetooth, Zigbee, microwave ovens and many other sources, so its occupancy is invisible to users and varies with place and time. This project builds a portable RF environment monitor that captures 2.4 GHz energy, converts it to a calibrated power reading in dBm, tags it with latitude/longitude/altitude, detects interference bursts and jamming with machine learning, and visualises everything live and on a map.

Limitation: the logarithmic detector measures total in-band power. It is not a spectrum analyser and cannot identify individual channels or protocols.

1.2 Core engineering objectives

1. **RF front-end acquisition** with antenna, LNA and AD8318 detector.
2. **Edge digitisation and location tagging** with ESP32 ADC and NEO-7M GPS.
3. **Cloud ingestion and REST services** over Wi-Fi/HTTPS with Bearer-token authentication.
4. **Unsupervised anomaly detection** from rolling power statistics, SNR and rate of change.
5. **Interactive single-page visualisation** of live charts, history, events and signal map.
6. **Physics simulation testbench** with 6 repeatable scenarios for validation before hardware.

2. COMPONENTS & TECHNOLOGIES

2.1 Hardware bill of materials

No.	Component	Model / specification	Supply	Cost (INR)
1	Microcontroller	ESP32-WROOM-32 (30-pin DevKit)	3.3 V / 5 V VIN	450
2	RF power detector	AD8318 module (1 MHz–8 GHz, 60 dB)	5.0 V	1,200
3	Low-noise amplifier	SPF5189Z (+13.7 dB at 2.4 GHz)	5.0 V	650
4	Antenna	2.4 GHz 50 Ω SMA omni (3 dBi)	passive	250
5	GPS receiver	u-blox NEO-7M with ceramic antenna	3.3 V	850
6	Display	0.96" SSD1306 I ² C OLED, 128 $\times$ 64	3.3 V	220
7	Battery	18650 Li-ion, 3.7 V, 2600 mAh	3.0–4.2 V	280
8	Charger & protection	TP4056 micro-USB with DW01A	5.0 V USB	45
9	Boost regulator	MT3608 step-up, set to 5.0 V	3.0–4.2 V in	75
10	Supporting hardware	SMA adapters, jumpers, enclosure	—	350
	Total			4,370

2.2 Software stack

Layer	Technology
Firmware	Arduino C++ / ESP-IDF; TinyGPSPlus; SSD1306 driver; Wi-Fi station with non-blocking reconnection
Backend	Python 3.12, Flask 3.0 (blueprint /api), Gunicorn WSGI, hosted on Render
Database	SQLite (local) and PostgreSQL (cloud) through a dual-engine layer
Device layer	Hardware abstraction (device/interface.py) with simulator and ESP32 drivers
Signal processing	NumPy: rolling EMA/moving average (window 20), rate of change, SNR
Machine learning	Pure-Python/NumPy Isolation Forest, 100 trees (ml/isolation_forest.py); predictor pipeline (ml/predict.py)
Frontend	React 19, Vite 8, Recharts 3.10, Leaflet + OpenStreetMap; hosted on Vercel
Testing	Automated simulator tests (tests/test_simulator_physics.py)
Documentation	LaTeX report; build instructions file

3. HARDWARE ARCHITECTURE AND CIRCUIT DIAGRAM

Circuit diagram: RF front-end, ESP32, GPS, OLED and power chain

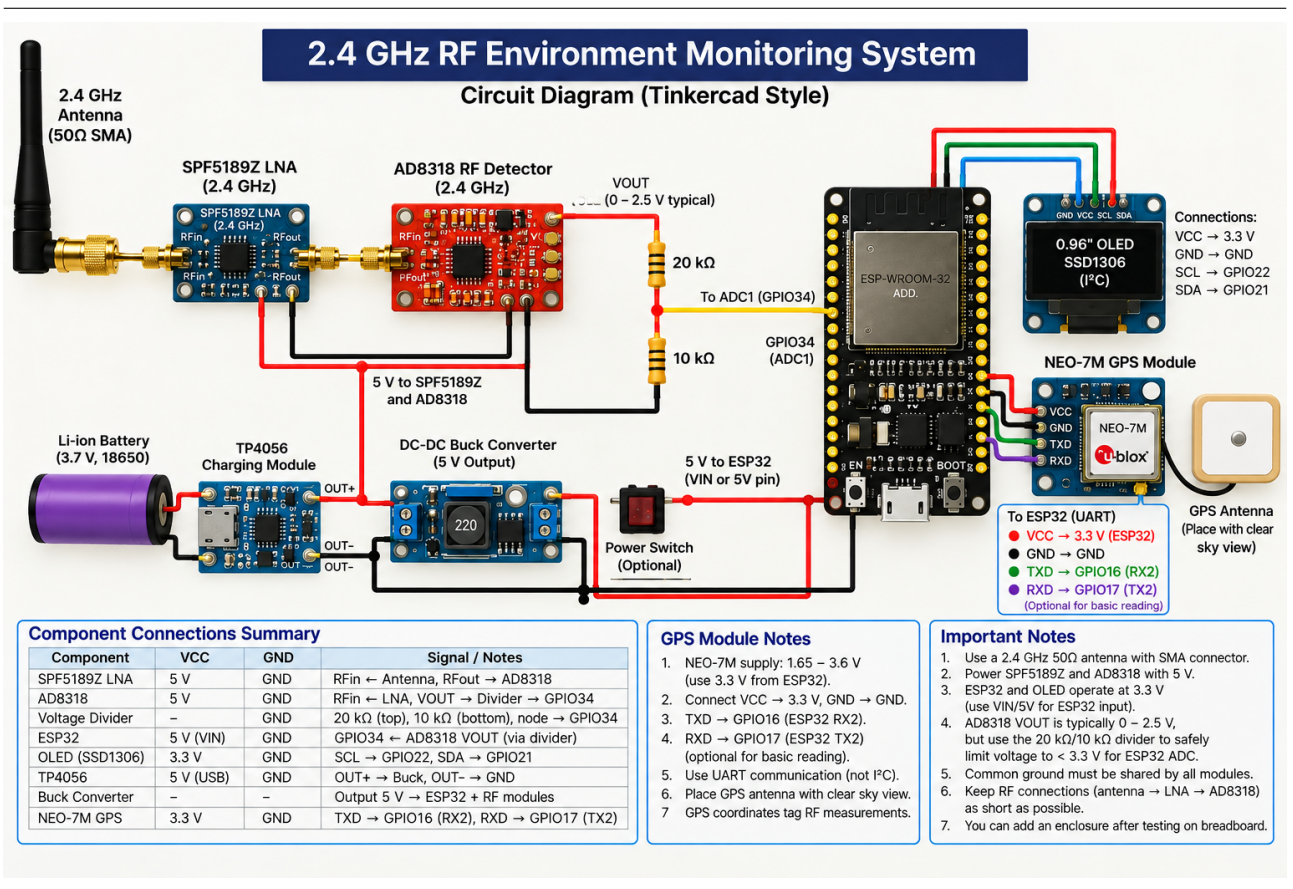


Figure 1: Circuit design drawing. Note: the power block in this drawing shows a buck converter; the plan replaces it with an MT3608 boost converter (Sec. 16).

Signal path and power distribution

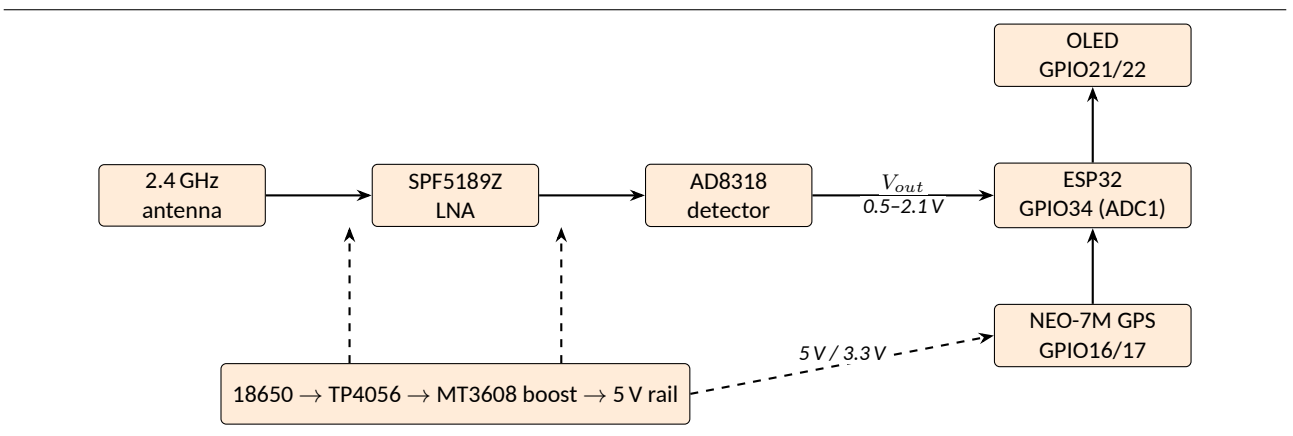


Figure 2: Signal path (solid) and 5 V power distribution (dashed), following the plan.

3.1 Hardware components, purpose and pin connections

No.	Component	Purpose in this project	Pins / connections
1	2.4 GHz 50 Ω SMA antenna (3 dBi omni)	Receives broadband 2.4 GHz ISM energy (Wi-Fi, Bluetooth, Zigbee, microwave-oven leakage) from the surroundings.	SMA connector $\rightarrow$ SPF5189Z RFin. Passive, no supply.
2	SPF5189Z low-noise amplifier	Adds about +13.7 dB gain at 2.4 GHz so weak signals reach the detector's input range, with low added noise.	RFin $\leftarrow$ antenna; RFout $\rightarrow$ AD8318 RFin; VCC $\leftarrow$ +5 V (about 90 mA); GND $\leftrightarrow$ common ground.
3	AD8318 logarithmic RF power detector module	Converts total in-band RF power (−60 to 0 dBm) into a DC voltage that varies logarithmically with power.	RFin $\leftarrow$ SPF5189Z RFout; VOUT $\rightarrow$ ESP32 GPIO34 (ADC1_CH6), 0.50–2.10 V, directly or via the 20 k Ω /10 k Ω divider; ENBL $\leftarrow$ 3.3 V (enable); VCC $\leftarrow$ +5 V (about 68 mA); GND.
4	ESP32-WROOM-32 (30-pin DevKit)	Edge controller: digitises the detector voltage (12-bit ADC, 64-sample burst average), reads GPS, drives the OLED and sends JSON over Wi-Fi/HTTPS.	GPIO34 $\leftarrow$ AD8318 VOUT (analog input only); GPIO16 (RX2) $\leftarrow$ NEO-7M TXD; GPIO17 (TX2) $\rightarrow$ NEO-7M RXD; GPIO21 (SDA) and GPIO22 (SCL) $\leftrightarrow$ OLED; 3V3 output for OLED, GPS and ENBL; VIN $\leftarrow$ +5 V rail; GND.
5	u-blox NEO-7M GPS with ceramic antenna	Gives latitude, longitude, altitude and time to tag every RF reading for the coverage map.	VCC $\leftarrow$ 3V3 (1.65–3.6 V, about 35 mA); GND; TXD $\rightarrow$ GPIO16 (RX2), 9600 baud NMEA-0183; RXD $\leftarrow$ GPIO17 (TX2), optional configuration.
6	0.96" SSD1306 I ² C OLED (128 $\times$ 64)	Local readout of power (dBm), noise floor and Wi-Fi status without the dashboard.	VCC $\leftarrow$ 3V3; GND; SDA $\leftrightarrow$ GPIO21; SCL $\leftarrow$ GPIO22.
7	18650 Li-ion cell (3.7 V, 2600 mAh)	Portable energy source for field measurements.	+ / − $\rightarrow$ TP4056 B+ / B− (through the DW01A protection).
8	TP4056 charger with DW01A protection	Charges the cell from micro-USB and protects it against over-charge, over-discharge and short circuit.	IN+ / IN− $\leftarrow$ 5 V micro-USB; B+ / B− $\leftrightarrow$ 18650; OUT+ / OUT− $\rightarrow$ MT3608 VIN / GND.
9	MT3608 step-up (boost) regulator, set to 5.0 V	Raises the 3.0–4.2 V cell voltage to the regulated +5 V main rail; a buck converter cannot do this.	VIN+ / VIN− $\leftarrow$ TP4056 OUT; VOUT+ $\rightarrow$ +5 V rail (ESP32 VIN, SPF5189Z VCC, AD8318 VCC); VOUT− $\rightarrow$ GND.
10	1000 μ F low-ESR capacitor	Absorbs ESP32 Wi-Fi current bursts and prevents brownout.	Across the +5 V rail and GND, close to the ESP32 VIN pin.
11	20 k Ω / 10 k Ω resistor divider (option)	Scales the detector output to below 3.3 V to protect the ADC input.	20 k Ω from AD8318 VOUT to node; 10 k Ω from node to GND; node $\rightarrow$ GPIO34.
12	Power switch, SMA adapters, jumpers, enclosure	Switches the battery supply, joins the RF chain, wires the modules and shields and protects the unit.	Switch in series between MT3608 output and the +5 V rail; SMA adapters between antenna, LNA and detector.

3.2 Pin interconnection

Module	Pin	ESP32 / rail	Characteristics / purpose
AD8318	VOUT	GPIO34 (ADC1_CH6)	Analog 0.50–2.10 V; raw detector voltage
AD8318	ENBL	3.3 V	Digital high; enables detector
AD8318	VCC / GND	+5 V / GND	5.0 V, about 68 mA
SPF5189Z	VCC / GND	+5 V / GND	5.0 V, about 90 mA
SSD1306	SDA, SCL	GPIO21, GPIO22	I ² C, 3.3 V
SSD1306	VCC / GND	3V3 / GND	Logic power
NEO-7M	TXD	GPIO16 (RX2)	UART 9600 baud, NMEA-0183
NEO-7M	RXD	GPIO17 (TX2)	UART configuration
NEO-7M	VCC / GND	3V3 / GND	3.3 V, about 35 mA
MT3608	OUT+ / OUT–	VIN / GND	Regulated +5.0 V main rail

3.3 Subsystem summary: hardware front-end

Purpose	Convert 2.4 GHz power to a voltage the ESP32 can digitise safely; supply position and power.
Current implementation	Full BOM, pin map and power chain are defined (Week 1). Circuit drawing exists; the physical circuit is not yet fully assembled.
Status / verification	Week 1 requirements/BOM/datasheet audit: Completed . Breadboard wiring and multimeter validation: In progress . Power bench test (Week 3) and front-end validation (Week 4): Planned .
Dependencies	Antenna, LNA, detector, ESP32 ADC, GPS, OLED, boost power rail.
Current issues	Buck vs. boost mismatch in the drawing (plan fixes with MT3608); AD8318 transfer function polarity to be checked (Sec. 16); ESP32 Wi-Fi current spikes need a 1000 μ F bypass capacitor.
Next action	Week 3: build and test the 5 V rail (± 0.05 V at 350 mA). Week 4: confirm $V_{out} \leq 2.5$ V and ADC input safety.

4. WORKFLOW

4.1 End-to-end data flow

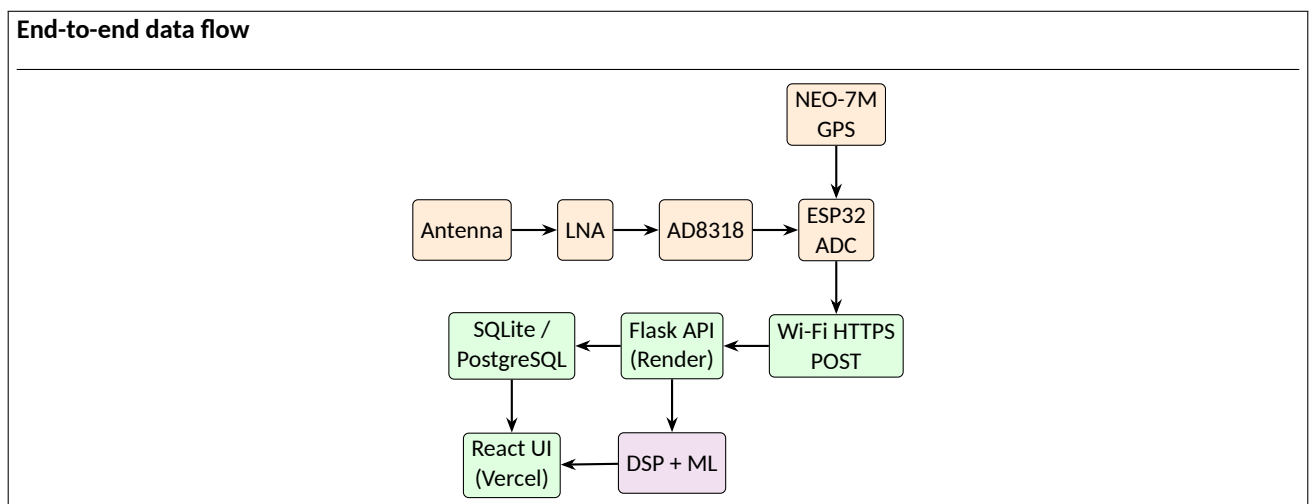


Figure 3: Hardware path from antenna to dashboard. The simulator feeds the same backend (Sec. 11).

4.2 Signal processing chain

Each measurement passes through: 64-sample burst average $\rightarrow$ ADC count to V_{det} $\rightarrow$ transfer function to dBm $\rightarrow$ calibration offset $\rightarrow$ 20-sample moving average, SNR and rate of change $\rightarrow$ Isolation Forest $\rightarrow$ alerts, database and dashboard. Alert rules: power > -30 dBm, SNR < 5 dB, anomaly score ≥ 0.60 . The planned transfer function in the firmware is $P_{in}(\text{dBm}) = -60.0 - 40.0 \times (V_{det} - 0.50)$; its polarity and intercept are to be confirmed against the AD8318 datasheet and bench calibration (Weeks 4–5, Sec. 16).

5. ARCHITECTURE DIAGRAMS

5.1 Layered system architecture

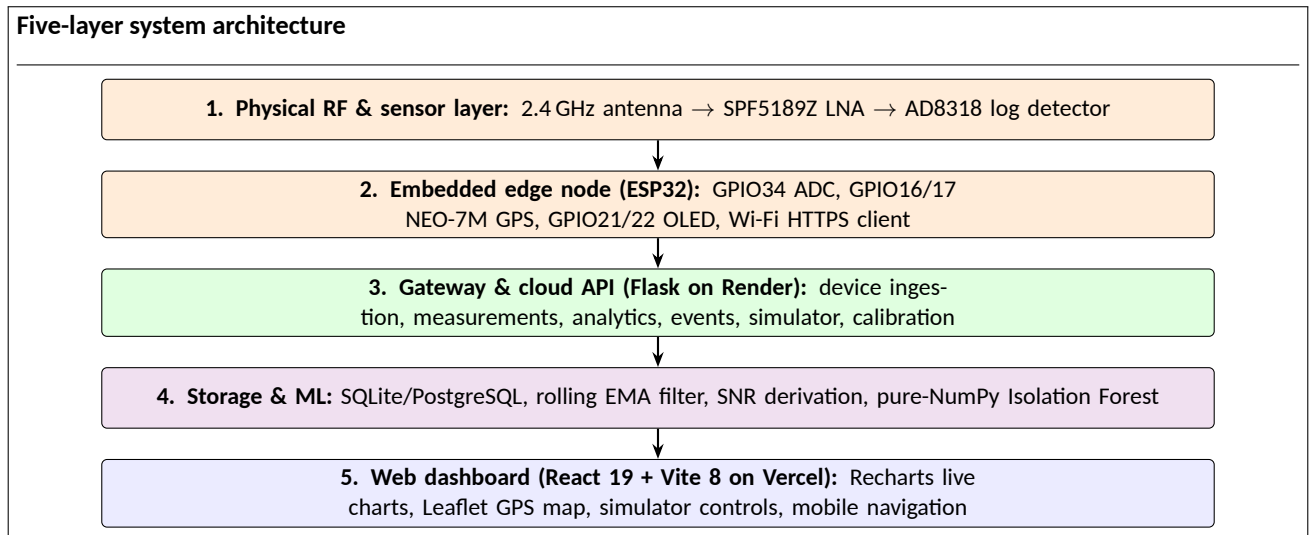


Figure 4: Layered architecture as defined in the master plan.

5.2 Frontend and backend architecture with technology stack

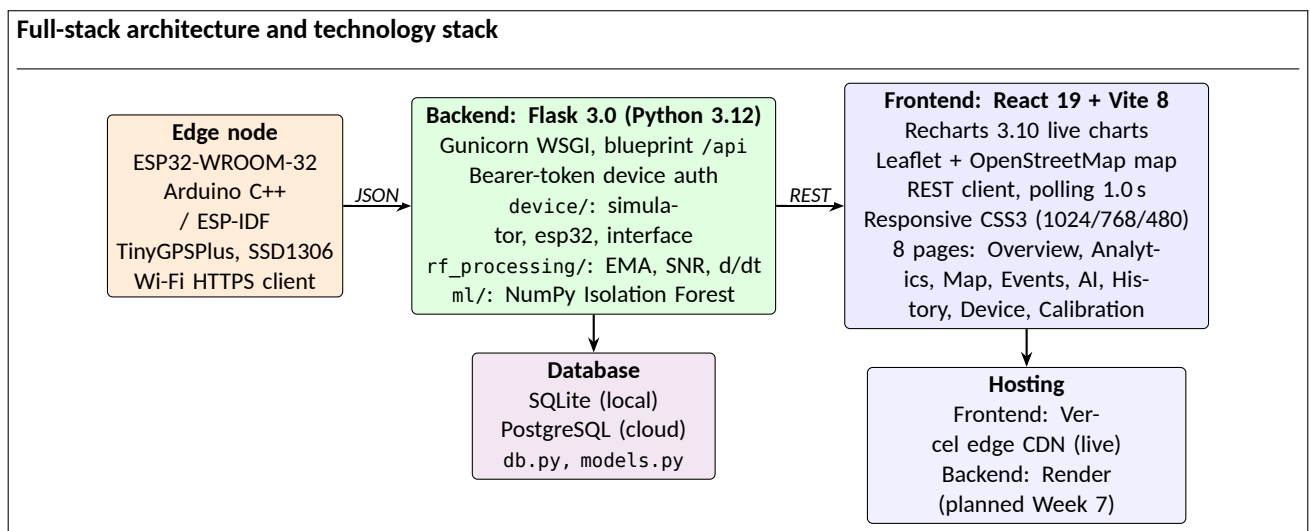


Figure 5: Edge, backend and frontend layers with the technology used in each.

5.3 Software module interaction

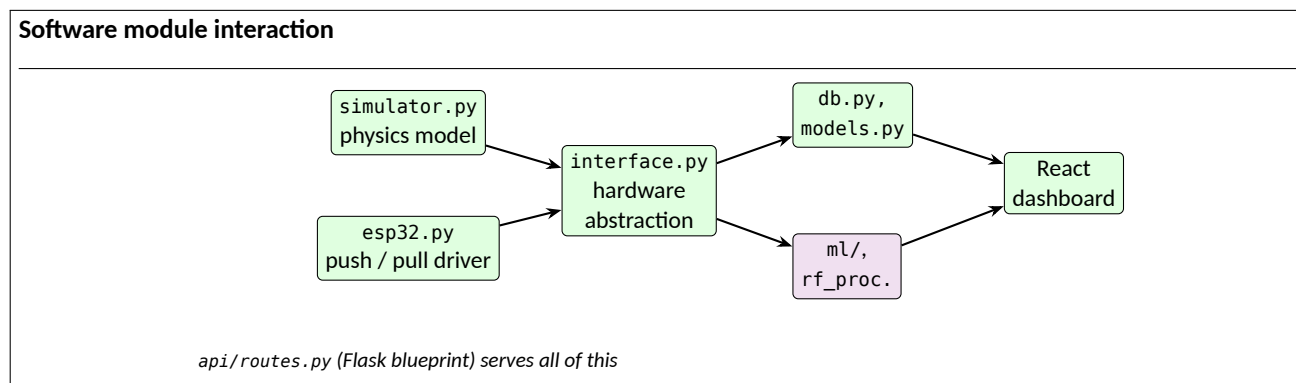


Figure 6: Both data sources use one abstraction, so the dashboard is unaware of the source.

6. EMBEDDED FIRMWARE AND DEVICE WORKFLOW

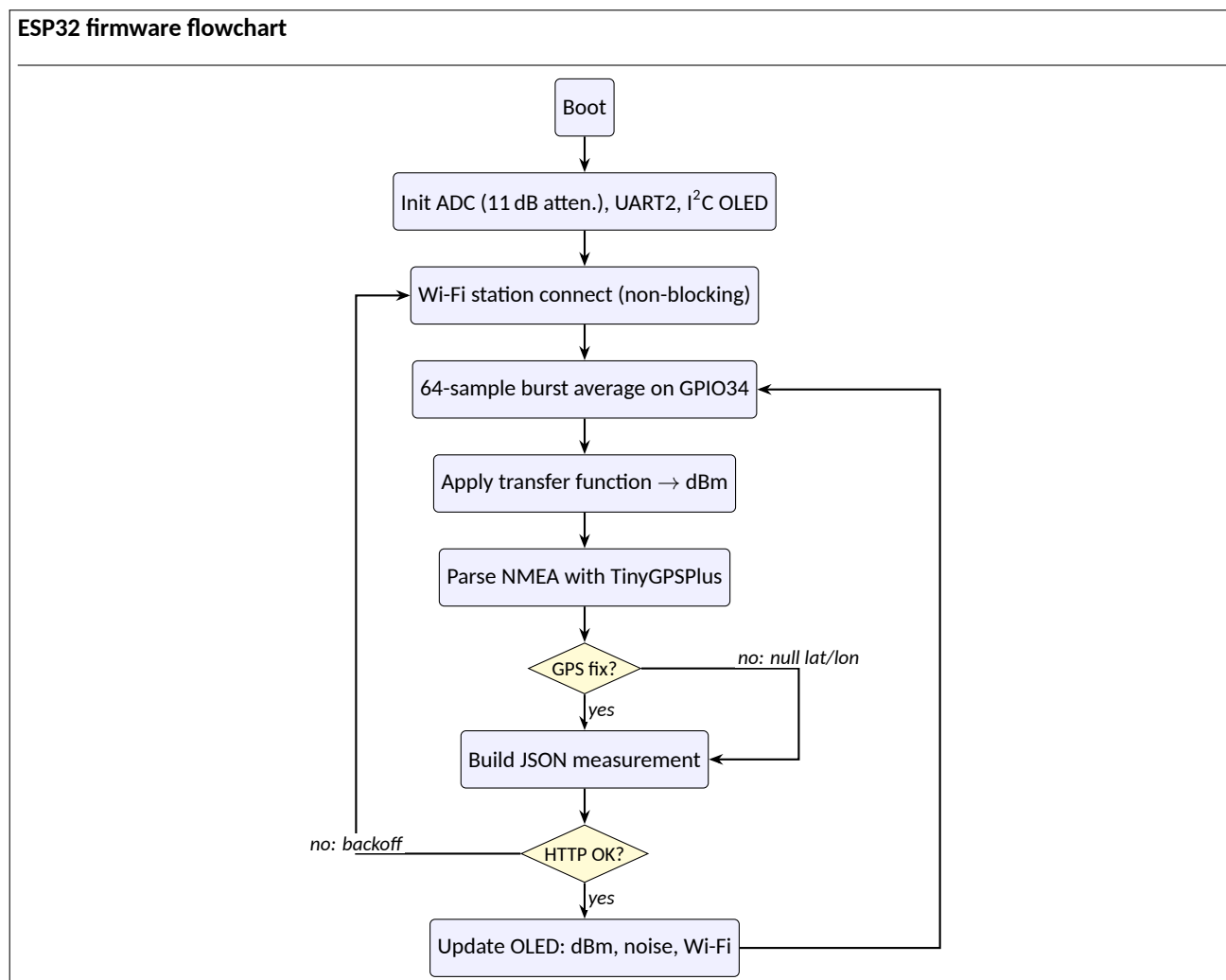


Figure 7: Planned firmware loop (Weeks 5–6).

6.1 Subsystem summary: firmware

Purpose	Digitise the detector voltage, read GPS, display locally and transmit JSON to the cloud.
Current implementation	Design and algorithms defined in the plan (burst averaging, transfer function, NMEA parsing, reconnect backoff). The device-side server code and the ESP32 driver in the backend (device/esp32.py) are implemented for both push and pull modes.
Status / verification	Firmware sketch: Planned (Week 5). GPS and telemetry client: Planned (Week 6). Backend side of the ESP32 driver: Completed per plan audit.
Dependencies	Stable 5 V rail (Week 3), safe ADC input (Week 4), backend ingestion route.
Current issues	Firmware not yet written; ADC non-linearity requires calibration offsets.
Next action	Week 5: ADC1_CH6 sampling with OLED readout. Week 6: GPS tagging and HTTPS POST to /api/device/measurements.

7. BACKEND AND API ARCHITECTURE

7.1 Pipeline

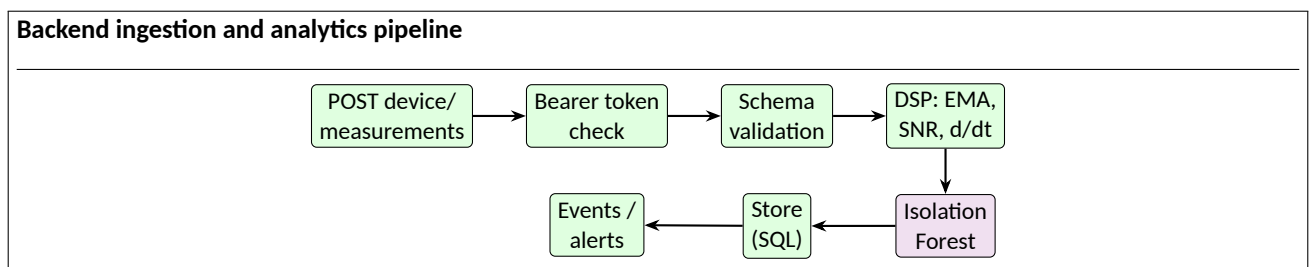


Figure 8: Backend processing for each incoming measurement.

7.2 REST endpoints

Endpoint	Method	Purpose
/api/device/measurements	POST	Ingest ESP32 telemetry with Authorization: Bearer <TOKEN>
/api/measurements/latest	GET	Latest RF measurement with active ML prediction
/api/measurements/history	GET	Rolling history (?minutes=60&limit=2000)
/api/analytics/timeseries	GET	Downsampled series: sig, noise, snr, vdet, adc
/api/analytics/summary	GET	Min, max, mean, median, standard deviation
/api/events	GET	Alerts (POWER_HIGH, POWER_LOW, SNR_LOW, POWER_SPIKE) and anomalies
/api/simulator/config	GET/POST	Read or tune distance, wall loss, interference
/api/simulator/scenario	POST	Apply a preset scenario
/api/calibration	GET/POST	Record and adjust reference power offsets

7.3 Subsystem summary: backend

Purpose	Receive measurements, authenticate devices, compute statistics, run ML, persist and serve the dashboard.
Current implementation	Flask application factory (app.py), blueprint with 16 active endpoints (api/routes.py), thread-safe hardware abstraction (device/interface.py), an auto-recovery check for the live stream thread under Gunicorn.
Status / verification	Local implementation: Completed and recorded as verified in the plan audit (built ahead of Week 7). Cloud deployment on Render, Bearer-token enforcement and PostgreSQL: Planned (Week 7).
Dependencies	Database layer, simulator and ESP32 drivers, ML module.
Current issues	Not yet exercised with real hardware packets.
Next action	Week 7: deploy to Render with Gunicorn, enable PostgreSQL and token authorisation.

8. DATABASE ARCHITECTURE

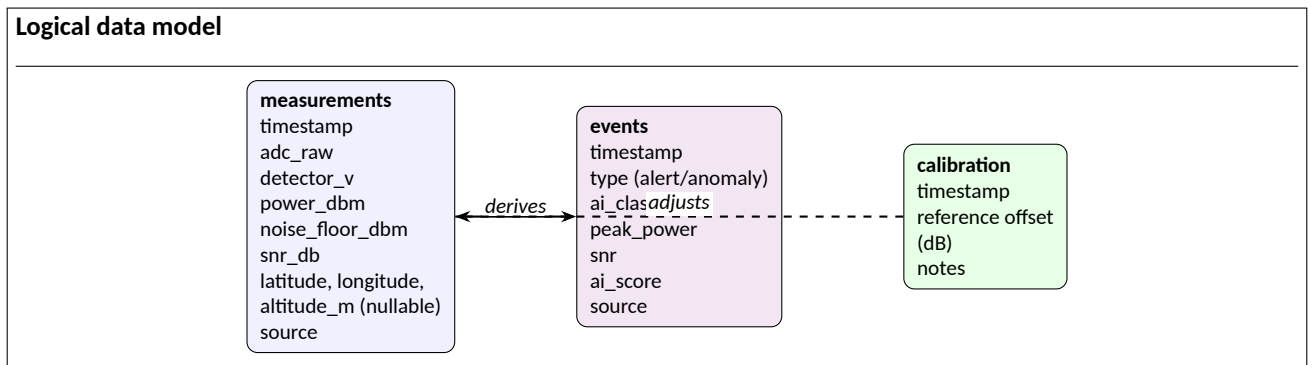


Figure 9: Logical schema inferred from API fields and UI; exact column names are in `database/models.py`.

Purpose	Persist measurements, events and calibration; support history and time-window queries.
Current implementation	Dual-engine layer (<code>database/db.py</code> , <code>models.py</code>): SQLite locally, PostgreSQL on cloud. Null GPS coordinates are accepted so RF telemetry is never dropped.
Status / verification	Local SQLite: Completed (plan audit). PostgreSQL on Render: Planned (Week 7).
Dependencies	Backend writer; timestamp ordering for history queries.
Current issues	Retention policy not yet defined.
Next action	Week 7: run the schema on PostgreSQL and confirm a 60-minute history query.

9. RF PROCESSING AND MACHINE LEARNING

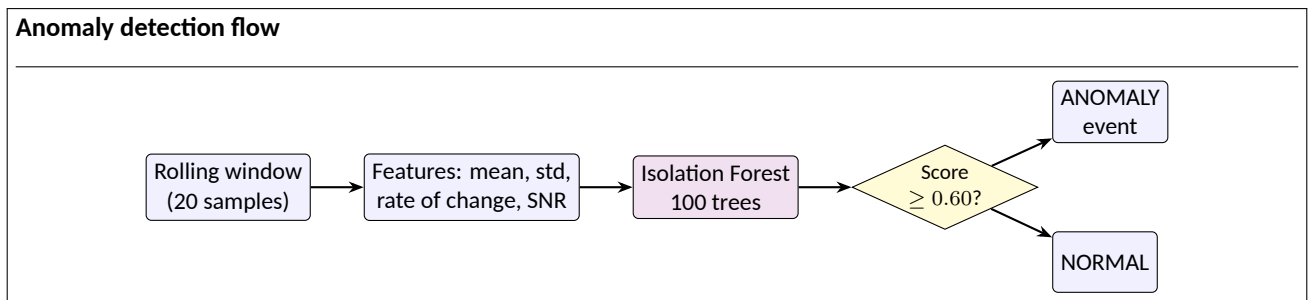


Figure 10: Inference flow. Rule alerts (power, SNR) run in parallel with the ML score.

Why a pure-NumPy model: Windows AppLocker/WDAC blocked scikit-learn's compiled DLLs on the development machine, so the Isolation Forest was re-implemented in pure Python/NumPy (`ml/isolation_forest.py`); `ml/predict.py` wraps it with a scaler pipeline.

Observed in the current simulated dashboard (screenshots, 9 Oct): 5-minute window of 558 samples with signal max 0.00 dBm and mean -38.14 dBm, noise floor mean -95.00 dBm, SNR mean 56.85 dB, and recent events all labelled ANOMALY with scores near 0.67.

Purpose	Convert detector voltage to dBm, estimate noise floor and SNR, flag abnormal RF activity.
Current implementation	Moving-average/EMA filters, rate of change, SNR (rf_processing/signal_processing.py); Isolation Forest and predictor; alert rules for power > −30 dBm, SNR < 5 dB and score ≥ 0.60.
Status / verification	Implementation: Completed per plan audit (built ahead of Week 8). Validation on real RF data: Planned (Weeks 10–11, target false-positive rate < 5%).
Dependencies	Calibrated dBm values, representative training/baseline data, simulator realism.
Current issues	Simulated data shows a 0.00 dBm maximum and constant ANOMALY labels, so the simulator parameters or threshold need tuning; the noise-floor chart axis labels look wrong; SNR of 50–95 dB is unrealistic for a −95 dBm floor.
Next action	Week 8: review thresholds and simulator noise; verify automatic alerts. Week 11: benchmark detection with microwave oven and Wi-Fi bursts.

10. FRONTEND DASHBOARD ARCHITECTURE

Pages: Monitor (Overview, RF Analytics, Coverage Map, RF Events), Analysis (AI Analysis, History), System (Device, Calibration). Header controls: live/pause, refresh rate (default 1.0 s), ONLINE and SIMULATED badges. **Live deployment:** <https://rf-signal-monitoring-system.vercel.app> (the page loads; its title still reads “AD8317”, to be aligned with the AD8318 used in the plan). Layout is responsive at breakpoints 1024, 768 and 480 px with a mobile drawer.

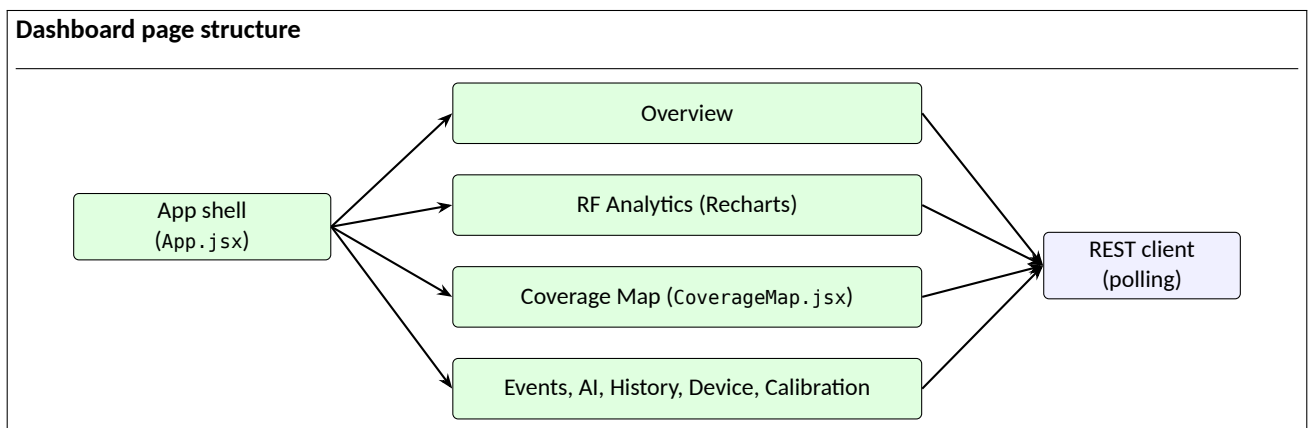


Figure 11: Dashboard structure.

Overview page showing RF power, noise floor, SNR, detector voltage, ADC value and AI status (simulated data)

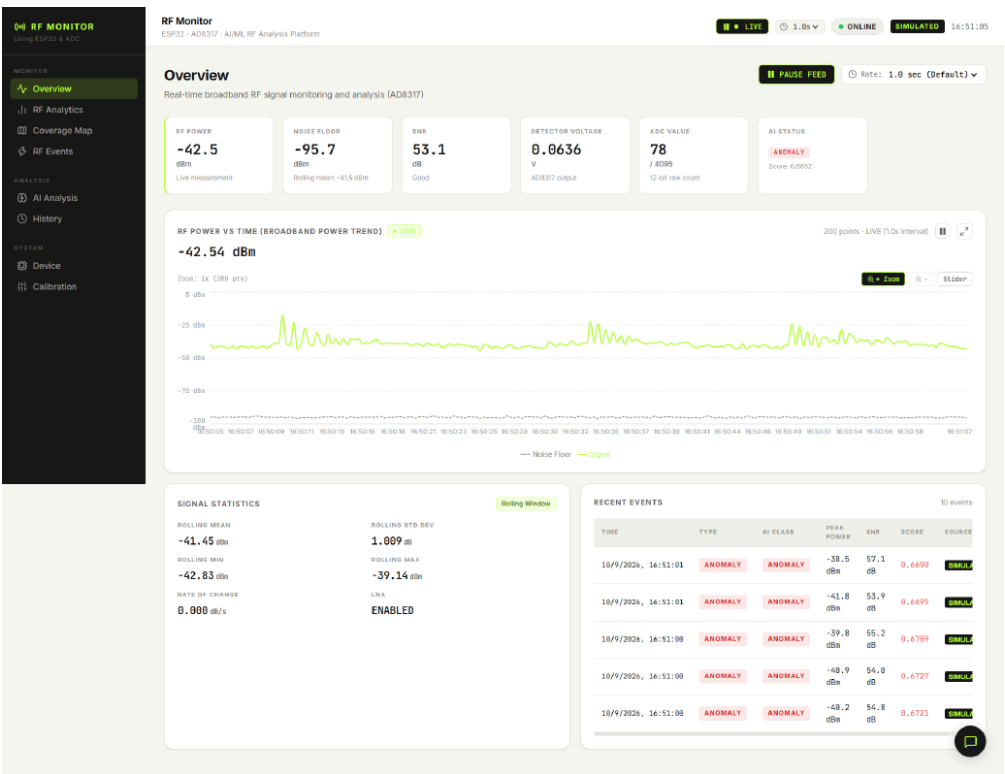


Figure 12: Overview page showing RF power, noise floor, SNR, detector voltage, ADC value and AI status (simulated data)

RF Analytics page: 5-minute window, 558 samples (simulated data)

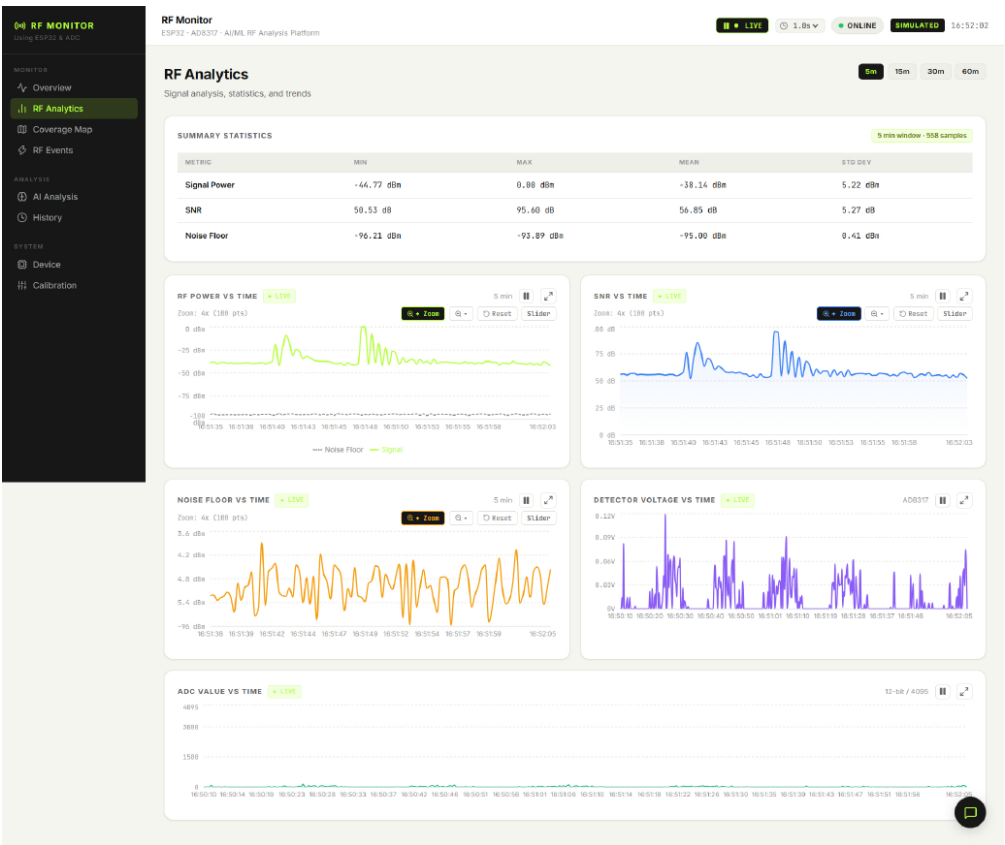


Figure 13: RF Analytics page: 5-minute window, 558 samples (simulated data)

Coverage Map waiting for GPS-tagged data (simulated demo location)

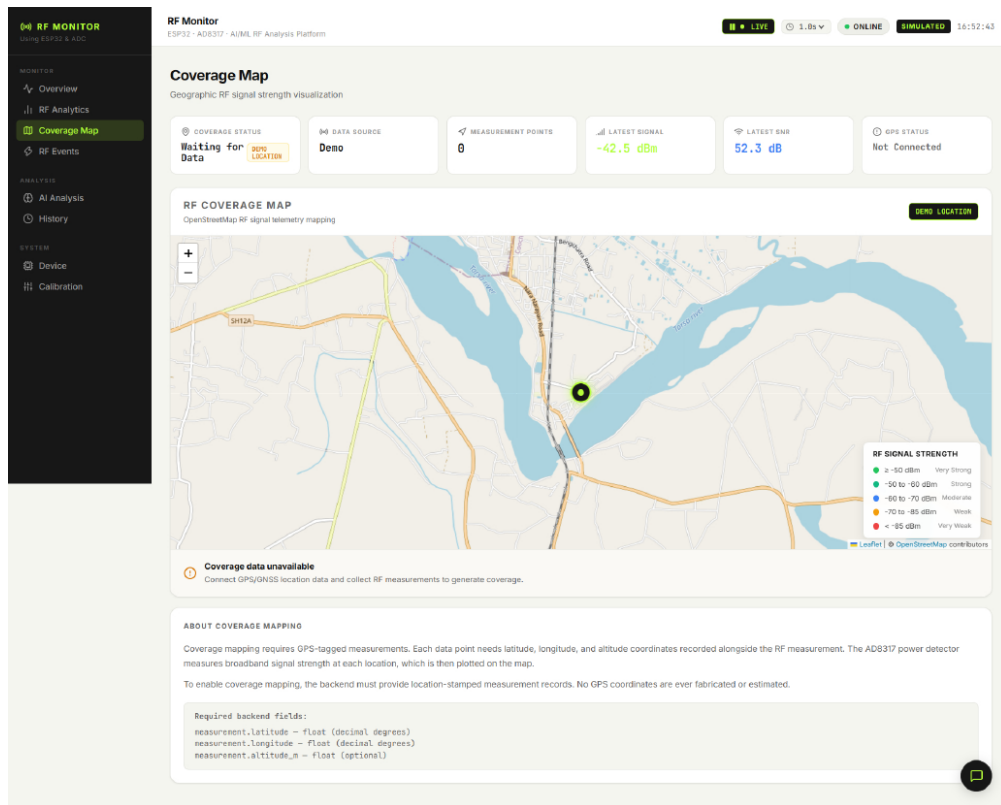


Figure 14: Coverage Map waiting for GPS-tagged data (simulated demo location)

Purpose	Show live and historical RF data, AI results, events and GPS coverage on desktop and mobile.
Current implementation	React 19 / Vite 8 SPA with Recharts live charts, zoom and time windows, Leaflet/OSM coverage map, pause/resume control and responsive layout.
Status / verification	Implemented and recorded as verified in the plan audit (Vite production build in 518 ms); screenshots above confirm Overview, Analytics and Coverage pages running on simulated data. Deployed on Vercel at the live link above (Completed); sub-second update and mobile checks against the cloud backend: Planned (Week 9).
Dependencies	Backend endpoints; GPS-tagged records for the map.
Current issues	Coverage map is empty until real GPS records arrive; chart label issue noted in Section 9.
Next action	Week 9: connect the live site to the Render backend and confirm sub-second updates on mobile and desktop.

11. SIMULATION AND REAL-HARDWARE INTEGRATION

Simulator-to-hardware transition

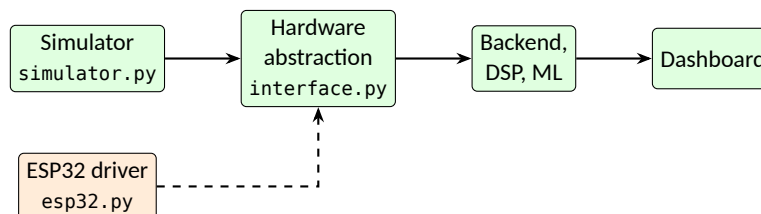


Figure 15: Both sources share one interface (dashed: real hardware, from Week 6).

11.1 Simulation scenarios

Scenario	Purpose and parameters
baseline	Line of sight, $d = 1$ m, no walls, noise floor -95 dBm, $P_{rx} \approx -40$ dBm
increased_distance	$d = 15$ m, $n = 2.0$, about 23.5 dB path loss
obstacle_introduced	$d = 5$ m, two concrete walls at 12 dB each (24 dB)
increased_interference	In-band co-channel interference at -68 dBm, SNR below 25 dB
device_outage	Transmissions stop; dashboard reports OFFLINE
recovery	Restores nominal conditions; alerts clear and graphs continue

11.2 Hardware bring-up stages

Stage	Goal	Plan week	Status
S0	Simulator drives full dashboard	Week 2	Completed
S1	Stable 5 V rail, safe analog input	Weeks 3–4	In progress
S2	ESP32 shows live dBm on OLED	Week 5	Planned
S3	GPS-tagged packets sent over Wi-Fi	Week 6	Planned
S4	Cloud backend accepts hardware stream	Week 7	Planned
S5	Physical RF hardware streams to cloud dashboard	Week 10	Planned
S6	Calibration and anomaly trials	Week 11	Planned

12. DEPLOYMENT ARCHITECTURE

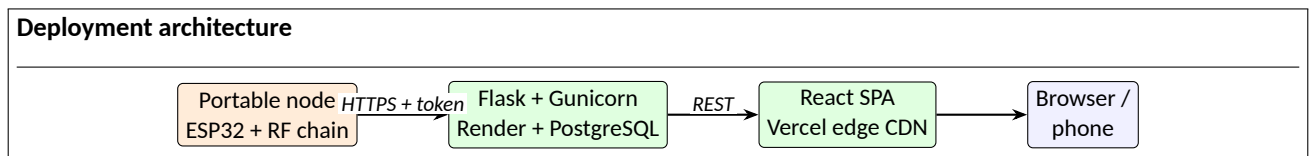


Figure 16: Deployment: frontend live on Vercel; backend on Render planned for Week 7; local development uses Flask with SQLite.

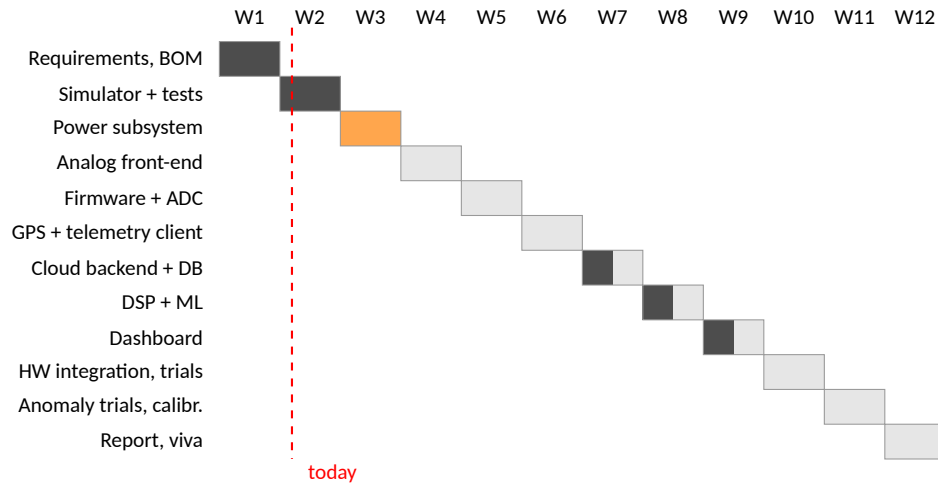
The frontend is already live on Vercel; the backend works locally and its Render/PostgreSQL deployment is scheduled for Week 7.

13. TESTING AND VALIDATION

Week	Item	Method	Pass criterion	Status
2	Simulator physics	30 automated tests	All pass	Completed
3	5 V power rail	350 mA dummy load	5.0 ± 0.05 V	Planned
4	Detector output safety	Multimeter, open circuit	$V_{out} \leq 2.5$ V	Planned
4	ADC input	11 dB attenuation; divider vs. direct	No overvoltage, linear	Planned
5	ADC and dBm readout	OLED vs. reference	Accurate live dBm	Planned
6	GPS and telemetry	Outdoor fix, HTTPS POST	Located packets arrive	Planned
7	API and auth	Bearer token, schema checks	Invalid rejected	Planned
8	Alert rules and ML	Injected scenarios	Alerts at set thresholds	Planned
9	Dashboard speed	Mobile and desktop	Sub-second updates	Planned
10	Path loss	1, 5, 15, 30 m outdoor; indoor walls	Log-distance trend	Planned
11	Anomaly detection	Microwave and Wi-Fi bursts	False positives $< 5\%$	Planned
11	Latency	RF event to dashboard	< 1.5 s	Planned

14. EXECUTION PLAN

12-week execution timeline (status as of 9 October 2026)



Dark = completed; orange = in progress; light = planned. Narrow dark bars in W7–W9 mark software already built ahead of schedule.

Figure 17: Gantt-style progress against the 12-week plan.

14.1 Month 1: Foundation and bench verification (Weeks 1–4, 1–28 Oct)

Week	Dates	Activities and milestone	Status
1	1–7 Oct	Finalise BOM, audit datasheets (AD8318, SPF5189Z, NEO-7M); all modules checked in lab inventory	Completed
2	8–14 Oct	Physics simulator (device/simulator.py) and 30 tests; 6 presets operational	Simulator and tests Completed ; week running
3	15–21 Oct	TP4056 + DW01A charging, MT3608 set to 5.00 V, 1000 μ F bypass capacitor; milestone: 5.0 ± 0.05 V at 350 mA	In progress (circuit assembly)
4	22–28 Oct	Detector open-circuit check (≤ 2.5 V), ADC 11 dB attenuation, divider vs. direct; milestone: no overvoltage risk	Planned

14.2 Month 2: Firmware, cloud and ML (Weeks 5–8, 29 Oct – 25 Nov)

Week	Dates	Activities and milestone	Status
5	29 Oct–4 Nov	ESP32 firmware, 64-sample ADC burst, transfer function, OLED display; milestone: live dBm on OLED	Planned
6	5–11 Nov	NEO-7M on UART2 with TinyGPSPlus, Wi-Fi reconnect, HTTPS client; milestone: located packets over Wi-Fi	Planned
7	12–18 Nov	Render deployment (Gunicorn), PostgreSQL, Bearer token; milestone: backend accepts hardware and simulator	Code ready (local); deployment Planned
8	19–25 Nov	Filters, SNR, Isolation Forest, alert triggers; milestone: scoring without external DLLs	Implemented (plan audit); threshold tuning Planned

14.3 Month 3: Integration, field trials and defence (Weeks 9–12, 26 Nov – 23 Dec)

Week	Dates	Activities and milestone	Status
9	26 Nov–2 Dec	Live charts, coverage map, mobile drawer, Vercel deployment; milestone: sub-second streaming	Implemented and live on Vercel; cloud-backend check Planned
10	3–9 Dec	Shielded enclosure; outdoor 1/5/15/30 m and indoor wall tests; milestone: hardware streams to cloud dashboard	Planned
11	10–16 Dec	Microwave and Wi-Fi interference trials, calibration offsets, latency below 1.5 s; false positives < 5%	Planned
12	17–23 Dec	Final report, viva slides, live demonstration	Planned

15. RISK ANALYSIS AND MITIGATION

ID	Risk	Severity	Mitigation
R-01	ESP32 ADC overvoltage (input > 3.3 V)	High	Verify $V_{out} \leq 2.5$ V; 11 dB attenuation; divider option
R-02	Brownout during Wi-Fi bursts	High	1000 μ F low-ESR capacitor on 5 V bus
R-03	Buck converter cannot make 5 V from a 3.7 V cell	High	Use MT3608 boost converter; update the circuit drawing
R-04	Detector calibration drift	Medium	Calibration offsets stored via /api/calibration
R-05	AppLocker/WDAC blocks scikit-learn DLLs	High	Pure-NumPy Isolation Forest (already implemented)
R-06	GPS lock lost indoors	Low	Null coordinates accepted; RF telemetry never dropped
R-07	Gunicorn kills stream thread	Medium	<code>_ensure_stream_alive()</code> recovery check (already added)
R-08	Wi-Fi disconnection	Medium	Non-blocking exponential reconnect backoff in firmware
R-09	Transfer-function polarity or intercept wrong	High	Verify against datasheet; bench calibration with a known source (Weeks 4–5)
R-10	Simulator unrealistic, misleading ML tuning	Medium	Tune simulator; re-evaluate thresholds on real data in Week 11

16. CURRENT PROGRESS, PENDING WORK, AND MENTOR REVIEW POINTS

16.1 Progress summary as of 9 October 2026

Area	Finished / in progress / planned
Requirements, BOM, datasheets	Completed (Week 1)
Simulator and 30 tests	Completed (Week 2)
Flask API (16 endpoints), database layer, hardware abstraction	Completed locally, ahead of schedule; cloud deployment Planned Week 7
DSP filters and Isolation Forest	Completed locally; tuning on real data Planned Weeks 8, 11
React dashboard (charts, map, mobile layout)	Completed ; live on Vercel (see link); cloud-backend link-up Planned Week 9
Physical circuit assembly	In progress (Week 3 task)
ESP32 firmware and OLED	Planned Week 5
GPS integration	Planned Week 6
Field trials and calibration	Planned Weeks 10–11
Final report and viva	Planned Week 12

16.2 Technical points for the mentor

1. **Power chain in the circuit drawing:** it shows a buck converter on a 3.7 V cell. The plan uses an MT3608 boost converter; the drawing should be corrected.
2. **Detector naming:** the circuit and BOM say AD8318 but the dashboard header says AD8317. Confirm the part and fix the UI label.
3. **Transfer function:** the planned formula $P = -60 - 40(V - 0.50)$ maps 0.5 V to -60 dBm, while the detector's output normally falls as power rises, so 0.5 V would correspond to the strong-signal end of the range. Please advise whether to verify polarity and intercept against the datasheet before firmware work.
4. **Simulator realism:** 0.00 dBm maximum, constant ANOMALY labels and unrealistic SNR need tuning.
5. **ADC choice:** is the ESP32 internal ADC with offsets acceptable, or should an external ADC be considered?
6. **Validation of ML:** how should detection be validated without labelled real data (microwave and Wi-Fi trials in Week 11)?

17. NEXT STEPS

1. Finish circuit assembly and the 5 V rail test (Week 3, by 21 Oct).
2. Complete front-end safety checks and ADC attenuation tests (Week 4, by 28 Oct).
3. Correct the circuit drawing (boost converter) and the AD8317/AD8318 label.
4. Start ESP32 firmware with ADC and OLED (Week 5).
5. Tune simulator and ML thresholds while hardware work proceeds.
6. Update this report at the end of each month with measured results.